

ECOM EXPERTS

Ecom compatible

Manual de usuario, técnico y de desarrollo

ecom_compatible 1.0.0 · 07/09/2026

Contenido

1. Para qué sirve

2. Instalación

3. Configuración

Objetivo del análisis

Consultas a fuentes externas

Publicar el histórico en GitHub

Cómo conseguir el token de GitHub

Depuración

4. Lanzar el análisis

Dejarlo trabajando solo (tarea programada)

5. Leer el informe

Agrupado por quién lo hizo

Dos cifras que no son lo mismo

Overrides

Veredicto por módulo

Severidades

Llamadas antiguas dentro de un if

Ver los cambios

6. Generar el fichero de cambios (.diff)

7. Overrides

8. Plantillas y tema

8 bis. Descargar los ficheros a tu ordenador

8 ter. Regenerar y reiniciar

9. Histórico

10. Publicar en GitHub

11. Plan de migración recomendado

12. Preguntas frecuentes

1. Qué hace

2. Origen de los datos

2.1 Resolución de herencia en el diff

3. Arquitectura

4. El analizador léxico

5. Control de falsos positivos

6. Base de datos

7. El histórico

8. Generación de parches

9. Fuentes remotas

10. Publicación en GitHub

11. Ejecución por lotes

11 bis. Tarea programada

11 ter. Generación de parches

11 quater. Carpeta protegida y descargas

11 quinquies. Regenerar y reiniciar

11 sexies. Clasificación por proveedor y estado

11 septies. Exclusión mutua

Tres reglas que vienen de un fallo real

Sin caché SQL

El lote nunca dura más que el cerrojo

11 octies. Coste del análisis

12. Seguridad

13. Limitaciones conocidas

1. Añadir una regla nueva

Función de PHP eliminada u obsoleta

Regla por expresión regular

Regla de plantilla

Sustitución recomendada

Reescritura mecánica (entra en los .diff)

2. Regenerar los mapas del núcleo

3. Probar sin una tienda

4. Reglas de estilo del módulo

5. Añadir un paso al análisis

console

6. Añadir una fuente remota

6 bis. Trampas de PrestaShop ya pisadas

7. Qué no hacer

8. Depuración

Saber, antes de actualizar, qué módulos van a romper la tienda y qué hay que cambiar exactamente.

1. Para qué sirve

Actualizar PrestaShop de 1.7 a 8.x o 9.x rara vez falla por el núcleo: falla por los módulos, los overrides y las plantillas del tema. Este módulo los revisa **antes** y te dice, para cada problema:

- Qué módulo lo tiene
- En qué fichero y en qué línea
- Qué orden concreta hay que cambiar y por cuál
- Qué pasa si no lo cambias

Además revisa si existe una versión más reciente del módulo ya compatible con tu destino.

2. Instalación

1. Sube la carpeta `ecom_compatible` a `/modules` de la tienda.
2. Panel de administración → **Módulos** → busca «Ecom Compatible» → **Instalar**.
3. Aparecerá una entrada nueva en **Parámetros avanzados** → **Ecom Compatible**.

No modifica nada de la tienda: solo lee ficheros y escribe en sus propias tablas.

3. Configuración

Módulos → Ecom Compatible → **Configurar**.

Objetivo del análisis

Opción	Qué hace
Versión de PrestaShop destino	La versión a la que quieres migrar. Determina el mapa de cambios que se usa.
Versión de PHP destino	PrestaShop 8.1 y superiores exigen PHP 8.1 como mínimo.
Analizar código / overrides / plantillas / tema	Puedes desactivar bloques para acelerar el análisis.
Generar propuestas de override	Escribe overrides adaptados en <code>generated/overrides/</code> . Nunca los instala solo.
Módulos excluidos	Separados por comas.
Módulos por lote	Bájalo a 1 o 2 si el análisis se corta a mitad.

Consultas a fuentes externas

Viene desactivado. Mientras esté en «No», el módulo no hace ni una sola petición a internet y todo el análisis es local.

Al activarlo consulta la API de PrestaShop, el Marketplace de Addons y GitHub para saber si hay versiones superiores compatibles. Puedes añadir un token de GitHub (sin él, GitHub limita a 60 consultas por hora) y declarar repositorios de módulos propios con el formato `mimodulo=usuario/repositorio`, uno por línea.

Publicar el histórico en GitHub

Viene desactivado. Sube el histórico acumulado a un repositorio tuyo, para llevar la memoria de todas las tiendas que auditas.

Antes de activarlo: publicar saca información de la tienda hacia GitHub. Deja «**Anonimizar**» en **Sí** salvo que el repositorio sea privado. Las opciones «Incluir ficheros .diff» e «Incluir overrides generados» publican **código real de tus módulos**: no las actives con un repositorio público.

Cómo conseguir el token de GitHub

GitHub no permite que una aplicación externa capture tu sesión: hay que generar un token y pegarlo una vez. Son dos minutos. Las instrucciones también aparecen dentro de la propia configuración del módulo, con el enlace directo.

1. Crea el repositorio en GitHub si no existe. **Puede estar vacío** (el módulo lo inicializa solo) y **puede ser privado**.
2. Abre <https://github.com/settings/personal-access-tokens/new>
3. Ponle nombre («Ecom Compatible») y elige caducidad.
4. En **Repository access** marca «Only select repositories» y elige tu repositorio.

5. En **Permissions** → **Repository permissions** busca **Contents** y ponlo en **Read and write**. No hace falta nada más.
6. Pulsa **Generate token** y copia el valor que empieza por `github_pat_`. Solo se muestra una vez.
7. Pégallo en «Token con permiso de escritura», guarda y pulsa «**Probar la conexión con GitHub**». Debe decir que la conexión es correcta y que hay permiso de escritura.

Alternativa más rápida: un token clásico con el permiso `repo` desde <https://github.com/settings/tokens/new?scopes=repo>

Si usas GitHub CLI en tu ordenador: el comando `gh auth token` imprime un token ya válido que puedes pegar directamente.

El token se guarda en la configuración de la tienda y solo se envía a `api.github.com`. Si se filtra, revócalo desde la misma página donde lo creaste.

Depuración

Si algo falla, activa «Guardar logs», repite la operación y envía el fichero de `modules/ecom_compatible/logs/`.

4. Lanzar el análisis

Parámetros avanzados → **Ecom Compatible** → **Analizar la tienda**.

Salvo que actives la tarea programada (ver más abajo), el análisis solo se ejecuta cuando pulsas ese botón.

La **consola en vivo** aparece solo cuando hay algo que contar: al analizar, al regenerar, al empaquetar, al reiniciar, al publicar o cuando algo falla. Mientras no ejecutes nada, no ocupa sitio. Muestra cada fichero que analiza, las plantillas, los overrides, la consulta remota y el veredicto de cada módulo. Si algo falla, el error sale ahí en rojo con la respuesta literal del servidor: es lo que hay que enviarme para diagnosticar.

El trabajo se reparte en lotes por AJAX **fichero a fichero**, y cada lote se corta por tiempo (8 segundos por defecto, y nunca más del 60 % del límite del servidor). Por eso da igual que un módulo tenga 20 ficheros o 5.000: no puede agotar el tiempo de ejecución.

Si ves un recuadro rojo que dice «El JavaScript del módulo no se ha cargado», los botones no van a responder. Comprueba que existe `modules/ecom_compatible/views/js/admin.js` y que se puede leer.

Dejarlo trabajando solo (tarea programada)

Para catálogos muy grandes puedes dejar que trabaje solo durante días:

1. En la configuración, activa «**Permitir la tarea programada**», guarda con el token vacío y se generará uno automáticamente.
2. Copia la URL que aparece en ese mismo bloque.
3. Programa una llamada cada 10-15 minutos en tu servidor:

```
*/15 * * * * wget -q -O /dev/null "https://tutienda.com/index.php?
fc=module&module=ecom_compatible&controller=cron&token=TU_TOKEN"
```

Cada llamada trabaja unos segundos, guarda el progreso y sale. Reanuda siempre donde lo dejó y solo empieza un análisis nuevo si el anterior terminó hace más de 20 horas (añade `&start=1` para forzarlo). Puedes abrir la URL en el navegador para ver el avance en texto plano.

Con «**Generar los .diff al terminar**» activado, al acabar escribe un parche por módulo. Con «**Publicar en GitHub al terminar**» además lo sube. Esa es la única situación en la que el módulo envía algo sin que pulses un botón, así que actívala solo si es lo que quieres.

5. Leer el informe

Agrupado por quién lo hizo

El informe no es una lista plana: separa los módulos por origen, porque **lo que hay que hacer con un módulo roto depende de quién lo mantiene**.

Sección	Qué significa	Qué hacer
PrestaShop (nativos)	Los distribuye PrestaShop	Nada. Se actualizan solos al actualizar el núcleo
De la plantilla	Vienen con tu tema	Actualizar el tema a una versión compatible, o cambiar de tema. No los parchees a mano: la siguiente actualización se llevaría los cambios
Propios	Los tuyos	Aquí es donde aplican directamente los <code>.diff</code> y los overrides generados
De terceros	De otros proveedores	Buscar versión compatible, escribir al desarrollador o sustituir el módulo
Sin autor declarado	No declaran autor	Abre el fichero principal y mira la cabecera
Instalados pero desactivados	No se ejecutan	No romperán nada, pero el actualizador los toca igual. Desinstálalos antes de migrar si no los vas a usar
No instalados (solo carpeta)	Hay carpeta pero no están en la tienda	Peso muerto. Borra la carpeta antes de migrar

Cada sección lleva su recuento propio de bloqueantes y avisos, y una columna **Desarrollador**.

Todo es plegable y viene cerrado —las secciones de módulos y también los paneles de Overrides, Histórico, Ficheros generados, Reiniciar y Publicar— para que con 200 módulos veas de un golpe el mapa completo y abras solo lo que te interese. Arriba tienes un índice con enlaces directos (al pulsar, la sección se

abre sola) y los botones **Abrir todo** / **Cerrar todo**. El módulo recuerda qué dejaste abierto, así que al volver a la página sigue como lo dejaste.

Dos cifras que no son lo mismo

- **Módulos bloqueados**: cuántos módulos tienen al menos un problema que rompe en la versión destino.
- **Códigos bloqueantes**: cuántas órdenes concretas hay que cambiar. Un solo módulo puede tener decenas.

Por eso verás cosas como «7 módulos · 3 bloqueados · 62 códigos bloqueantes»: son tres magnitudes distintas. Lo que te dice el tamaño del trabajo es la segunda y la tercera juntas; lo que te dice si puedes actualizar es la segunda.

Dentro de una sección con varios proveedores —típicamente «De terceros» y «No instalados»— los módulos se **subdividen por desarrollador**, ordenados de más a menos bloqueantes. Así ves de un vistazo qué proveedor te está dando más problemas, que es como se gestiona esto: proveedor a proveedor, no módulo a módulo.

Cómo lo averigua: primero la lista de módulos que distribuye PrestaShop (69 nombres extraídos de las distribuciones reales 1.7.8, 8.1 y 8.2 y de la API oficial); después las dependencias que el tema declara en su `theme.yml` y los módulos que tiene dentro; después tus autores propios; y **si nada de eso encaja, el campo `author` del `config.xml` o de la clase principal**. Los módulos de PrestaShop que se instalan aparte (`paypal` , `ps_accounts` , `ps_mbo` , `gamification` ...) se detectan por autor.

En la pestaña **Ejecución** de la configuración puedes ajustar «Autores propios» para que reconozca los tuyos.

Overrides

Tienen panel propio al final, agrupados por fichero. Son la causa más frecuente de que una tienda no arranque tras actualizar: si la clase o el método sobrescrito desaparece, el error es fatal y en cada petición.

Veredicto por módulo

Veredicto	Significado	Qué hacer
listo	Sin problemas detectados	Nada
revisar	Solo avisos	Míralos, pero no bloquean la actualización
actualizar	Existe una versión superior compatible	Actualiza el módulo antes de actualizar PrestaShop
bloqueado	Rompe en el destino	Corregir o desinstalar antes de migrar

Severidades

- **Bloqueante** — error fatal, pantalla en blanco o funcionalidad que desaparece en silencio. Hay que resolverlo.
- **Aviso** — cambio de comportamiento, algo que dejará de funcionar pronto, o una llamada antigua que está protegida por un `if`.
- **Informativo** — buenas prácticas y deuda técnica.

Llamadas antiguas dentro de un `if`

Un módulo bien escrito para varias versiones envuelve las llamadas antiguas en una comprobación:

```
if (version_compare(_PS_VERSION_, '8.0.0', '<')) {  
    Tools::generateIndex();  
}
```

Eso **no rompe nada** en la versión destino: esa rama sencillamente no se ejecuta. Cuando el módulo detecta una condición así — `version_compare`, comparaciones con `_PS_VERSION_` o `PHP_VERSION_ID`, o `class_exists` / `method_exists` / `function_exists` — baja el hallazgo de **bloqueante a revisar** y te dice en qué línea está la condición y qué dice exactamente, para que compruebes si cubre de verdad tu versión de destino.

Vale la pena mirarlos igualmente: si la condición está mal puesta, sí romperá; y si está bien, es código que va a dejar de ejecutarse y conviene limpiar.

Ver los cambios

Botón **Ver cambios** de cada módulo. Para cada problema aparece el título, la explicación de la consecuencia real, la instrucción concreta de qué escribir en su lugar, el fragmento de código afectado y el fichero con la línea.

6. Generar el fichero de cambios (.diff)

Botón **Generar .diff**. Produce un parche con las correcciones **mecánicas seguras**: renombrados y envoltorios eliminados que se pueden sustituir sin pensar (`Tools::jsonEncode(` → `json_encode(` , `Attribute::` → `ProductAttribute::` , `$texto{0}` → `$texto[0]` ...).

Se guarda en `modules/ecom_compatible/generated/informes/` con el nombre `modulo--vVERSION--psDESTINO.diff`, porque el mismo módulo puede existir con versiones distintas en tiendas distintas. Dentro, las rutas son las originales de tu tienda (`modules/mimodulo/fichero.php`), así que se aplica desde la raíz de PrestaShop con:

```
git apply modulo--v1.0.0--ps9.0.0.diff
```

Con la tarea programada activada y «Generar los .diff al terminar», se generan todos de golpe sin tocar nada.

Lo que exige criterio no entra en el parche: se queda en el informe para que lo decidas tú.

Haz siempre copia de seguridad antes de aplicar un parche, y pruébalo primero en una copia de la tienda.

7. Overrides

El módulo revisa los overrides de `/override` y los que traen los módulos, y detecta:

- Overrides de clases que ya no existen en el destino → la tienda no arranca
- Métodos sobrescritos que ya no existen → la funcionalidad desaparece, o error fatal si usan `parent::`
- Firmas incompatibles con el núcleo de destino → error fatal al cargar
- Dos módulos sobrescribiendo el mismo método → conflicto
- Overrides sin módulo propietario (restos de módulos desinstalados)
- Overrides con `namespace` o `declare(strict_types=1)`, que PrestaShop no puede cargar
- **Overrides que suplantán la clase del núcleo en vez de extenderla** → el fallo más silencioso de todos

Este último merece una explicación, porque es el que más caro sale. Un override es siempre `class Xxx extends XxxCore` dentro de `Xxx.php`. Si alguien copió la clase entera del núcleo de una versión antigua a `/override` —y pasa más de lo que parece—, el fichero declara `class XxxCore` y PrestaShop lo carga **en lugar** del núcleo: todos los métodos que las versiones posteriores hayan añadido dejan de existir, y la tienda contesta `Call to undefined method` en cada petición.

Caso real: un `override/classes/Link.php` copiado de PrestaShop 1.7.6 dejó una tienda actualizada a 8.2 con el back-office en error 500 permanente por `Link::getTabLink()`. Hasta esta versión, el análisis **saltaba ese fichero en silencio**: al no reconocer qué clase sobrescribía, no lo miraba.

En `override/modules/...` la regla es otra: ahí lo correcto es `class XxxOverride extends Xxx`, sin ninguna clase `...Core`, y no se marca como fallo.

Si activas «Generar propuestas de override», escribe versiones adaptadas en `modules/ecom_compatible/generated/overrides/`. **Son propuestas, no se instalan solas**. Los métodos que ya no existen quedan comentados con el código original dentro, para que puedas trasladar la lógica.

Para instalar uno: revísalo, cópialo a `/override/classes/`, y borra `var/cache/prod/class_index.php` y `var/cache/dev/class_index.php`.

8. Plantillas y tema

Se revisan las plantillas de los módulos y las del tema activo. Lo más habitual y lo que más problemas da:

El tema sobrescribe plantillas de un módulo. Cuando existe `themes/tu-tema/modules/algun-modulo/...`, esa copia congela la plantilla en el momento en que se copió. Si actualizas el módulo para que sea compatible con 9.x, el tema sigue sirviendo la versión antigua y anula la corrección. El informe te dice exactamente qué ficheros están en esa situación y cuál es el original.

También detecta sintaxis de PrestaShop 1.6 (`{$product->name}`, `{$HOOK_LEFT_COLUMN}`, `{$base_dir}` ...), que en 1.7 y superiores devuelve vacío, y llamadas a hooks que ya no existen.

8 bis. Descargar los ficheros a tu ordenador

Panel «**Ficheros generados**» del informe. Ahí tienes la lista de todo lo que ha producido el módulo, con tamaño y fecha:

Carpeta	Qué contiene
Parches .diff	Un fichero por módulo, <code>modulo--vVERSION--psDESTINO.diff</code>
Propuestas de override	Los overrides adaptados, listos para revisar
Informes JSON y Markdown	Los mismos datos que se publican en GitHub

Cada fila tiene su botón **Descargar**, y arriba hay un «**Descargar todo en ZIP**» que te lo baja de una vez. Dentro del ZIP los informes vienen con sus carpetas (`modulos/<módulo>/<version>/...`), no en un montón plano.

Todo se guarda en `modules/ecom_compatible/generated/` , **protegida con `.htaccess`** : nadie puede descargarla escribiendo la URL en el navegador, porque los informes llevan rutas y fragmentos de código de tu tienda. La única vía es esta página, que comprueba permisos y token.

Si el botón del ZIP te dice que falta `ZipArchive` , tu servidor no tiene esa extensión de PHP. Descarga los ficheros uno a uno; el resto funciona igual.

8 ter. Regenerar y reiniciar

Regenerar informes y .diff vuelve a crear todos los ficheros a partir del histórico que ya tienes, **sin repetir el análisis**. Úsalo cuando hayas cambiado la versión destino, cuando hayas borrado los ficheros sin querer, o cuando quieras los `.diff` de todos los módulos de golpe en lugar de uno a uno. Va por lotes y lo verás avanzar en la consola.

Reiniciar tiene un botón por cada cosa, para que no se lleve por delante lo que no quieres:

Botón	Qué borra	Qué conserva
Reiniciar análisis	El análisis en curso y todos los hallazgos	Histórico, ficheros generados, configuración
Borrar ficheros	Informes, <code>.diff</code> , overrides y ZIP	Todo lo demás (se recrean con Regenerar)
Vaciar caché	Las respuestas guardadas de Addons, PrestaShop y GitHub	Todo lo demás
Reiniciar todo	Análisis, hallazgos, ficheros y caché	El histórico y la configuración
Borrar histórico	La memoria acumulada de qué se detectó y cuándo se resolvió	Todo lo demás

Todos piden confirmación. El del histórico avisa aparte porque **no se puede deshacer**. Ninguno toca la configuración del módulo ni tus tokens.

9. Histórico

El histórico acumula todas las incompatibilidades encontradas y **sobrevive a la desinstalación del módulo**. Para cada una guarda cuándo se vio por primera vez, en qué versión del módulo, cuántas veces ha aparecido y si ya se resolvió.

Cuando actualizas un módulo y vuelves a analizar, las incompatibilidades que ya no aparecen se marcan como **resueltas** anotando en qué versión. Es el registro de qué arregló cada actualización.

Botón **Ver el histórico** en el informe. Para borrarlo, hay un botón en la configuración.

10. Publicar en GitHub

Con la publicación activada y el token configurado, dispones de dos botones:

- **Vista previa** — lista exactamente qué ficheros se subirían y su tamaño. No envía nada.
- **Publicar ahora** — pide confirmación y sube todo en un único commit.

Estructura que se crea en el repositorio:

README.md	Índice general
INDICE.json	Índice legible por máquina
modulos/<módulo>/README.md	Resumen y lista de versiones
modulos/<módulo>/<versión>/hallazgos.json	Datos completos
modulos/<módulo>/<versión>/hallazgos.md	Informe legible
modulos/<módulo>/<versión>/cambios-psX.diff	Parche (solo si lo activas)
overrides/<destino>/<Clase>.php	Overrides (solo si lo activas)

Cada versión del módulo tiene su propia carpeta. Si auditas la versión 1.0.0 en una tienda y la 1.2.0 en otra, conviven sin pisarse. Cada `hallazgos.json` guarda además desde qué versión de PrestaShop y de PHP se generó, para que sepas de dónde salió cada informe.

Cada publicación **acumula sobre lo anterior**: sobrescribe los ficheros de los módulos analizados y deja intactos los del resto. Así el repositorio va creciendo con cada tienda que audites, en lugar de reemplazarse. Si necesitas retirar entradas antiguas, bórralas desde GitHub.

Usa siempre la vista previa la primera vez.

11. Plan de migración recomendado

1. Configura el destino (versión de PrestaShop y de PHP).

2. Lanza el análisis con las consultas externas desactivadas.
 3. Resuelve los **bloqueantes** módulo a módulo, empezando por los que tengan más.
 4. Activa las consultas externas y vuelve a analizar: actualiza los módulos marcados como «actualizar».
 5. Vuelve a analizar. Los problemas resueltos aparecerán como tales en el histórico.
 6. Revisa los overrides y las sobrescrituras del tema.
 7. Repite hasta que no queden bloqueantes.
 8. Haz copia de seguridad completa y actualiza PrestaShop.
-

12. Preguntas frecuentes

¿Modifica mis módulos? No. Solo lee. Los `.diff` y los overrides se escriben dentro de su propia carpeta y los aplicas tú.

¿Envía información a algún sitio? No, salvo que actives las consultas externas (solo consulta versiones publicadas) o la publicación en GitHub (que además pide confirmación y trae vista previa). Ambas cosas vienen desactivadas.

Mi módulo es propio y no está en ningún sitio público. ¿Sirve igual? Sí: el análisis del código, los overrides y las plantillas es completamente local. Solo se pierde la comprobación de «existe una versión más nueva».

Sale «bloqueado» pero el módulo parece que funciona. Ahora funciona porque estás en la versión actual. El informe dice qué ocurrirá **en la versión destino**.

El análisis se corta a la mitad. Baja «Módulos por lote» a 1 o 2. Es el límite de tiempo de ejecución del servidor.

¿Puedo confiar en los datos? Los cambios del núcleo no están escritos a mano: se calculan comparando el código fuente real de PrestaShop 1.7.8.11, 8.1.7, 8.2.7 y 9.1.1. Aun así, revisa siempre antes de aplicar cambios en producción.

Ecom Compatible — Documentación técnica

Módulo: `ecom_compatible` · **Versión:** 1.0.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0 **Compatibilidad:** PrestaShop 1.7.0 → 9.x · PHP 7.0 → 8.4

1. Qué hace

Audita la tienda completa y responde a una pregunta: *si actualizo a PrestaShop 8.x o 9.x y a PHP 8.x, ¿qué se rompe, en qué fichero y en qué línea?*

Cubre cinco superficies:

Superficie	Qué revisa
Código PHP de los módulos	Clases, métodos, constantes y funciones que desaparecen o cambian de firma
Overrides	Los de <code>/override</code> y los que traen los módulos: clase base eliminada, método inexistente, firma incompatible, conflictos entre módulos, y ficheros que suplantan la clase del núcleo en vez de extenderla (<code>ovr_suplanta_nucleo</code> , <code>ovr_no_extiende</code>)
Plantillas <code>.tpl</code>	Sintaxis de PrestaShop 1.6, variables no asignadas, hooks inexistentes
Tema activo	Plantillas propias y sobrescrituras de plantillas de módulos que congelan versiones antiguas
Fuentes remotas	Distribution API, Marketplace de Addons y GitHub, para saber si existe una versión superior compatible

2. Origen de los datos

La precisión del módulo depende de no inventar nada. Los ficheros de `data/` **no están escritos a mano**: se generaron comparando el código fuente real de cuatro versiones de PrestaShop.

Fichero	Contenido	Cómo se generó
<code>core-diff-8.1.json</code>	Símbolos presentes en 1.7.8.11 y ausentes en 8.1.7	Tokenizado de <code>classes/</code> , <code>controllers/</code> , <code>src/Core/</code> , <code>src/Adapter/</code> de ambas versiones
<code>core-diff-8.2.json</code>	Igual, unión de 1.7.8.11 + 8.1.7 contra 8.2.7	Ídem
<code>core-diff-9.1.json</code>	Unión de 1.7.8.11 + 8.1.7 + 8.2.7 contra 9.1.1	Ídem
<code>core-classes-*.json</code>	Índice de nombres de clase existentes en cada destino	Ídem
<code>hooks-*.json</code>	Hooks eliminados, nuevos y alias	<code>install/data/xml/hook.xml</code> y <code>hook_alias.xml</code>
<code>rules-php.json</code>	Cambios de PHP 7.x → 8.x	Curado
<code>rules-smarty.json</code>	Reglas de plantillas	Curado
<code>rules-prestashop.json</code>	Reglas estructurales y de overrides	Curado
<code>rules-replacements.json</code>	Qué usar en lugar de cada símbolo eliminado	Curado
<code>rules-rewrites.json</code>	Reescrituras mecánicas seguras para los <code>.diff</code>	Curado

Cada diff se calcula contra la **unión** de todas las versiones anteriores, de modo que también se detecta lo que se añadió en 8.x y se eliminó en 9.x.

2.1 Resolución de herencia en el diff

Comparar clase contra clase, símbolo a símbolo, produce un falso positivo sistemático: si un método sube de la clase hija al padre, desaparece de la hija pero **la llamada sigue funcionando**. Reportarlo como eliminado manda al desarrollador a tocar código que no está roto.

Por eso el generador (`build_data.php`, fuera del módulo) aplica `resolverHeredado()` antes de escribir el diff: para cada método o propiedad que ha desaparecido de una clase, recorre la cadena `extends` en la versión destino —probando en cada nivel el nombre a secas y el sufijo `Core`, porque la convención legacy de PrestaShop es `Customer extends CustomerCore`— y si lo encuentra heredado, lo descarta.

El efecto medido sobre el destino 9.1 fue de 8 métodos y 17 propiedades retirados del diff (por ejemplo `Customer::validateController`, que sigue resolviendo por `ObjectModelCore`). El script `medir_herencia.php` cuantifica el residuo y hoy da **0 falsos positivos** sobre 391 métodos y 493 propiedades marcadas como eliminadas.

Los sustitutos de `rules-replacements.json` se validan aparte con `verificar_reglas.php`, que comprueba dos cosas contra el código real de 9.1: que el símbolo que damos por eliminado no existe, y que **el sustituto que proponemos sí existe**. Un sustituto inexistente es peor que no dar ninguno. Estado actual: 27 clases y 109 métodos, 0 incidencias.

Cifras del conjunto actual (destino 9.1): 302 clases eliminadas, 161 clases con métodos eliminados, 142 con firma cambiada, 10 hooks eliminados, 261 constantes y propiedades retiradas.

3. Arquitectura

```
ecom_compatible/
├── ecom_compatible.php           Clase Ecom_Compatible: install, tab, configuración
├── config.xml · logo.png · index.php
├── classes/
│   ├── EcomCompatLogger.php      Logs en /logs, condicionados por configuración
│   ├── EcomCompatSettings.php    Claves de configuración y valores por defecto
│   ├── EcomCompatDb.php          Tablas de trabajo + addNewFeatures()
│   ├── EcomCompatHistory.php     Histórico persistente (tabla propia)
│   ├── EcomCompatFinding.php     Recolector de hallazgos con volcado por lotes
│   ├── EcomCompatKnowledge.php   Carga y consulta de data/
│   ├── EcomCompatPhpScanner.php  Analizador léxico con token_get_all()
│   ├── EcomCompatInventory.php   Inventario de módulos, temas y overrides
│   ├── EcomCompatHttp.php        Cliente HTTP con caché en BD y límite de ritmo
│   ├── EcomCompatCoreAnalyzer.php Motor principal: cruza código con datos
│   ├── EcomCompatOverrideAnalyzer.php
│   ├── EcomCompatTemplateAnalyzer.php
│   ├── EcomCompatRemoteAnalyzer.php
│   ├── EcomCompatPatchBuilder.php Genera .diff en formato unificado
│   ├── EcomCompatOverrideBuilder.php
│   ├── EcomCompatExporter.php    Árbol de ficheros del repositorio
│   ├── EcomCompatGitHub.php      Publicación vía Git Data API
│   └── EcomCompatRunner.php       Orquestador por lotes
├── controllers/admin/AdminEcomCompatibleController.php
├── data/                          Base de conocimiento (JSON)
├── views/                         report.tpl · admin.css · admin.js
├── logs/                          Se crea al instalar
└── generated/                     overrides/ e informes/ generados
```

Sin `namespace` en la clase principal, sin dependencias de Composer, carga con `require_once`.

4. El analizador léxico

`EcomCompatPhpScanner` usa `token_get_all()` **sin** la bandera `TOKEN_PARSE`: funciona como lexer puro, así que puede analizar código con sintaxis de PHP 8 ejecutándose sobre PHP 7.1, y al revés, sin errores de compilación.

Extrae por fichero: namespace, importaciones `use`, clases (padre, interfaces, métodos con firma y línea, propiedades, escrituras dinámicas a `$this->`), llamadas estáticas `Clase::miembro`, instanciaciones, funciones llamadas y declaradas, constantes y literales de cadena.

Detalle crítico: la interpolación `"{$var}"` produce el token `T_CURLY_OPEN` (array) pero se cierra con un `}` literal (string). Sin contar esa apertura, el contador de llaves se desincroniza y el seguimiento de clases se rompe silenciosamente: durante el desarrollo esto hacía que se perdiera, entre otros, `Hook::exec()`. Está contemplado en `scanCode()` y en el generador de datos.

5. Control de falsos positivos

Un falso bloqueante cuesta más que un hallazgo omitido. Medidas aplicadas:

- **Clases propias del módulo:** se recogen en una primera pasada y se excluyen del contraste con el núcleo.
- **Clases legacy vs. con namespace:** para `XxxCore` se confía en el diff (calculado sobre código real); la comprobación cruzada contra el índice de clases solo se aplica a nombres con namespace, que pueden haberse movido.
- **Hooks:** solo se marcan como inexistentes los que aparecen en la lista explícita `removed` de `hooks-*.json`. Los hooks dinámicos (`actionObjectProductAddAfter`, `actionProductGridDefinitionModifier` ...) no están en `hook.xml` y marcarlos por ausencia generaría decenas de falsos positivos.
- **registerHook vs. unregisterHook:** se distinguen con `(?![A-Za-z])registerHook`.
- **Scripts de upgrade/:** se excluyen del inventario de hooks; describen versiones históricas, no el estado actual.
- **Módulos widget:** si la clase implementa `WidgetInterface` o declara `renderWidget()`, no se exige `hookDisplayX()`.
- **Ternario anidado:** el patrón excluye paréntesis, corchetes, llaves, comas y punto y coma entre los dos `?`, de modo que `($a ? $b : ($c ? $d : $e))` no salta.
- **Plantillas de administración:** las reglas del presentador de 1.7 (`{ $order->id }`, `{ $customer->firstname }`, `{ $base_dir }` ...) llevan `"context": "front"` y se saltan en `views/templates/admin/`. En el back office el controlador asigna objetos reales, así que ese código es correcto.
- **Guardas de versión y de existencia:** un módulo bien escrito para varias versiones envuelve las llamadas antiguas en un `if`. Eso cambia por completo la gravedad, y marcarlo como bloqueante hace perder la confianza en todo el informe:

```
php if (version_compare(_PS_VERSION_, '8.0.0', '<')) { Tools::generateIndex(); // no se ejecuta en 9.x: no rompe nada }
```

`EcomCompatPhpScanner::findGuardAbove()` recorre hacia atrás una ventana de 40 líneas buscando `version_compare`, comparaciones con `_PS_VERSION_` o `PHP_VERSION_ID`, y `class_exists` / `method_exists` / `function_exists` / `is_callable` / `property_exists`. Confirma contando llaves que la línea del hallazgo **sigue dentro** del bloque que abrió esa condición, de modo que un `if` ya cerrado no cuenta. Cubre además el caso de la guarda y la llamada en la misma línea, y la rama `else` de un `if` de versión. Se acepta también

una comprobación de existencia del símbolo concreto en cualquier punto del fichero, porque es un patrón defensivo habitual escribirla lejos del uso.

Cuando hay guarda, el hallazgo baja de **bloqueante a aviso**, cambia de regla (`core_class_removed_guarded` , `core_method_removed_guarded` , `php_function_removed_guarded`) y el texto cita la línea y el código exacto de la condición encontrada, para que se pueda comprobar a mano si cubre de verdad la versión destino.

La heurística es deliberadamente conservadora: en el peor caso rebaja algo que sí era grave, nunca al revés. `defined()` queda excluido a propósito, porque es la guarda de seguridad del principio del fichero y no una comprobación de API.

Verificado con seis escenarios: guarda envolvente, rama `else` , comprobación lejana, guarda en la misma línea, llamada sin guarda y llamada después de que el `if` se cerrara. Los seis se clasifican como corresponde. - **Anclaje de patrones:** las reglas que describen sentencias, como `declare(strict_types=1)` , se anclan a principio de línea; si no, saltan al encontrar el mismo texto dentro de una cadena literal. -

Repetición: máximo 8 ocurrencias por regla y fichero (6 en plantillas), indicando el total real. -

`installOverrides()` **no es un fallo**. La regla `ps_install_overrides` estuvo mal redactada: daba por hecho que instalar un override cuando el fichero ya existe destruye el del otro módulo, y recomendaba copiar a mano. Es falso, y el consejo era además dañino: copiar el fichero entero **sí** aplasta los overrides ajenos.

Leyendo `Module::addOverride()` en 9.1 se ve que PrestaShop hace una **fusión miembro a miembro**: reflexiona ambas clases e inserta método por método, propiedad por propiedad y constante por constante, etiquetando cada uno con un comentario `/* module / date / version */` para saber a quién pertenece. Solo lanza excepción cuando el **mismo** miembro ya está sobrescrito, y en ese caso nombra al módulo dueño. Dos módulos que sobrescriben métodos distintos de la misma clase conviven sin problema.

La regla se mantiene, con severidad **informativa**, para avisar del único caso real: el choque en el mismo método. La nota de `ovr_conflict` se corrigió en el mismo sentido.

Verificación empírica:

Conjunto	Resultado
9 módulos oficiales de PrestaShop 1.7.8	0 bloqueantes
Módulo con incompatibilidades inyectadas	17 bloqueantes , todos reales
9 módulos propios de terceros	4 bloqueantes, los 4 confirmados leyendo el código
El propio <code>ecom_compatible</code> contra 9.1 / PHP 8.4	0 bloqueantes

La detección de ternario anidado se contrastó contra el propio parser de PHP 8.3, que señala la misma línea.

6. Base de datos

Tabla	Ciclo de vida	Contenido
<code>ecomcompat_scan</code>	Se borra al desinstalar	Estado del análisis por lotes: paso, cursor, cola, estadísticas
<code>ecomcompat_finding</code>	Se borra al desinstalar	Hallazgos del análisis: ámbito, módulo, severidad, regla, fichero, línea, fragmento, detalle, solución
<code>ecomcompat_module</code>	Se borra al desinstalar	Fila resumen por módulo: contadores, estado remoto, veredicto
<code>ecomcompat_cache</code>	Se borra al desinstalar	Caché de respuestas HTTP con TTL
<code>ecomcompat_history</code>	Sobrevive a la desinstalación	Histórico acumulado con huella única, primera y última vez visto, apariciones, estado y versión de resolución

Se conservan los 5 análisis más recientes (`EcomCompatDb::pruneScans`).

No hay ficheros `upgrade-x.y.z.php` : `addnewfeatures()` se ejecuta al abrir la configuración y al entrar en el informe, crea lo que falte y añade columnas nuevas con `ALTER TABLE` dentro de `try/catch`.

7. El histórico

`EcomCompatHistory` mantiene una entrada por incompatibilidad con huella

`md5(módulo|destino|regla|fichero|título)` . La línea **no** entra en la huella: un desplazamiento del código no crea una entrada nueva.

- Primera detección → alta con `first_seen` y `first_version` .
- Detección repetida → incrementa `occurrences` , actualiza `last_seen` y la línea.
- Deja de detectarse en un módulo ya analizado → `status = resolved` y se anota `resolved_version` .
- El estado `ignored` se conserva entre análisis.

Consecuencia práctica: al reinstalar el módulo o al actualizar un módulo auditado, el histórico dice qué se arregló y en qué versión.

8. Generación de parches

`EcomCompatPatchBuilder` produce diffs en formato unificado sin dependencias externas: recorte de prefijo y sufijo comunes, subsecuencia común más larga sobre el bloque restante, agrupación en bloques con 3 líneas de contexto.

Solo incluye las reescrituras de `rules-rewrites.json` : sustituciones 1 a 1 que no cambian la semántica

(`Tools::jsonEncode(` → `json_encode(` , `Attribute::` → `ProductAttribute::` , `${0}` → `${s[0]}` , `"${v}"` → `"${v}"` ...). Todo lo que exige criterio se queda como hallazgo, nunca en el parche.

Dos detalles que hacían fallar `git apply`:

1. **Salto de línea final.** `preg_split` deja un elemento vacío al final; contarlo hacía que la cabecera `@@` declarase una línea de más. Resuelto en `splitLines()`.
2. **Frontera por la izquierda.** La sustitución de `Attribute::` convertía `ProductAttribute::` en `ProductProductAttribute::`. Se antepone `(?![A-Za-z0-9_\\$>])`, pero **solo** cuando el patrón empieza por carácter de nombre: para `->ajaxDie()` esa frontera sería incorrecta y anularía la coincidencia.

Los parches generados se validan con `git apply --check`.

9. Fuentes remotas

Réplica de la lógica del módulo oficial `autoupgrade` v7.6.5:

- **Distribution API** `https://api.prestashop-project.org/modules/{versión}` → módulos nativos de esa versión · `/prestashop` → rango de PHP por versión
- **Marketplace** `https://api.addons.prestashop.com/v2/products/{nombre_técnico}` Se recorre `technical_info.releases[]` y se selecciona la versión con `compatible_from <= destino <= compatible_to`, igual que `MarketplaceService::findCompatibleModuleUpgrade()`.
- **GitHub** `https://api.github.com/repos/{owner}/{repo}/releases` Repositorio deducido para módulos `ps_*` y los nativos conocidos; el resto se declara a mano en la configuración.

Salvaguardas: interruptor general desactivado de fábrica, caché en BD con TTL configurable, límite de peticiones por lote, `verify_peer` activo, tiempo de espera configurable.

Almacén de certificados. Muchos alojamientos compartidos dejan `curl.cainfo` y `openssl.cafile` vacíos en `php.ini`; entonces cURL falla en toda petición HTTPS con `unable to get local issuer certificate` (errno 60). En lugar de rebajar la verificación SSL —que nunca se desactiva— `EcomCompatHttp::getCaBundle()` busca el paquete de certificados que distribuye PrestaShop en `vendor/composer/ca-bundle/res/cacert.pem` y lo pasa por `CURLOPT_CAINFO`. Verificado: el fallo se reprodujo en el entorno de desarrollo y el respaldo lo resolvió.

10. Publicación en GitHub

`EcomCompatGitHub` usa la **Git Data API** (blobs → tree → commit → ref) para subir todo el árbol en un único commit, en lugar de un commit por fichero como haría la Contents API.

Repositorio recién creado. Un repositorio sin ningún commit rechaza *toda* la Git Data API con HTTP 409 `Git Repository is empty`: ni siquiera se pueden crear blobs. Por eso, cuando la rama no existe, `bootstrapEmptyRepo()` siembra el primer commit con la **Contents API** (`PUT /contents/README.md`), que sí funciona sobre un repositorio vacío, y después continúa el flujo normal. Comprobado contra un repositorio privado real recién creado: dos commits, el de inicialización y el del contenido.

Con historia previa se usan `base_tree` y `parents`, de modo que cada publicación es un commit incremental sobre el anterior. La rama se codifica por segmentos para que nombres como `release/9` no conviertan la barra en `%2F`.

Verificado contra un repositorio privado real recién creado: 119 ficheros publicados en un único commit, y una segunda publicación encadenada correctamente sobre la primera.

Salvaguardas, en este orden:

1. `PUBLISH_ENABLED` desactivado de fábrica.
2. Token de escritura introducido a mano; sin él no se hace nada.
3. `PUBLISH_ANONYMIZE` activado de fábrica: no se publican fragmentos de código ni rutas absolutas.
4. Los `.diff` y los overrides generados —que **sí** contienen código real de la tienda— son opciones aparte, desactivadas de fábrica.
5. Modo simulación que lista ruta y tamaño de cada fichero sin enviar nada.
6. Confirmación explícita en el navegador antes de publicar de verdad.
7. Límite de ficheros por publicación.
8. Nunca se ejecuta desde un hook, un cron ni la carga de una página.

11. Ejecución por lotes

El análisis se reparte en lotes que el navegador pide por AJAX, o que llama la tarea programada. El estado vive en `ecomcompat_scan`, nunca en la sesión.

```
prepare → modules → overrides → theme → patches → finish → publish → done
```

Granularidad por fichero, no por módulo. Un módulo de terceros puede tener miles de ficheros; procesarlo entero en una petición agotaría `max_execution_time`. El bucle es:

```
while (tiempo consumido < presupuesto) { procesar el siguiente fichero }
```

Cuando se agota el presupuesto se guardan `cursor_index` (módulo) y `file_cursor` (fichero dentro del módulo) y se devuelve el control. El presupuesto se toma de la configuración y **nunca supera el 60 % del `max_execution_time` real** del servidor (`EcomCompatSettings::timeBudget()`), así que la petición siempre termina antes de que PHP la corte.

Lo que sobrevive entre lotes. La lista de ficheros **no** se guarda: `listFiles()` devuelve siempre el mismo orden alfabético, así que el cursor numérico sigue siendo válido y se recalcula en cada lote. Guardarla supondría escribir y releer un blob de varios MB por petición en un módulo grande. Lo que sí viaja en la columna `state` es el acumulador del módulo: clases propias, hooks registrados, hooks declarados y banderas de estructura, todo ello deducible solo del conjunto y no de un fichero suelto.

Las clases propias se recogen con una expresión regular (`EcomCompatInventory::collectDeclaredClasses`) en lugar de tokenizando: solo hace falta el nombre, y esa pasada previa tiene que ser barata.

Consola. Cada lote devuelve un array `console` con lo que ha hecho: fichero analizado, plantillas revisadas, overrides, consulta remota, veredicto del módulo. El navegador lo va volcando en un panel con marca de tiempo. Si la respuesta no es JSON —token caducado, falta de permisos, error de PHP— se vuelca la respuesta literal a esa misma consola, que es la única forma de diagnosticar el fallo sin acceso al servidor.

Detector de JS ausente. La plantilla pinta un aviso rojo que `admin.js` elimina nada más cargarse. Si el aviso sigue visible, el JavaScript no se ha cargado. Es el fallo más difícil de ver a ojo: la página se pinta con normalidad y los botones sencillamente no responden.

Tiene dos causas posibles, ambas silenciosas y ambas ya resueltas:

1. `addCSS()` y `addJS()` traen `$check_path = true` por defecto: PrestaShop resuelve la URI contra el sistema de ficheros y, si no la encuentra, **descarta el asset sin emitir ningún aviso**. El módulo los llama con `$check_path = false`.
2. El back office imprime los `js_files` en `admin/themes/*/template/header.tpl`, o sea **dentro de `<head>`**, antes de que exista el contenido de la página. Un script que arranque ahí encuentra `getElementById('ecomcompat') === null` y aborta con un `return` temprano: el fichero se ha cargado, pero no engancha ningún botón y el aviso se queda puesto, que es justo el caso que más confunde. Por eso toda la inicialización vive en `boot()` y se invoca con `DOMContentLoaded` cuando `document.readyState === 'loading'`.

Al terminar el arranque, `boot()` escribe «Módulo cargado y listo» en la consola: si esa línea aparece, el JavaScript está vivo.

11 bis. Tarea programada

`controllers/front/cron.php` expone un punto de entrada que reanuda el mismo análisis por lotes, de modo que un catálogo de miles de ficheros se puede recorrer a lo largo de varios días.

```
index.php?fc=module&module=ecom_compatible&controller=cron&token=<token>
```

- **Nombre de clase:** el despachador compone `<nombre_modulo> . <controlador>` . `'ModuleFrontController'`, es decir `ecom_compatiblecronModuleFrontController`. Como los nombres de clase de PHP no distinguen mayúsculas, la declaración `Ecom_CompatibleCronModuleFrontController` resuelve correctamente.
- **Autenticación:** token de 16 a 128 caracteres, generado al azar al guardar la configuración con el campo vacío, comparado con `hash_equals()` para no filtrar información por el tiempo de comparación. Sin `CRON_ENABLED` y sin token válido devuelve 403.
- **Reanudación:** si hay un análisis con `status = running`, lo continúa. Solo empieza uno nuevo si el último terminó hace más de 20 horas, o si se pasa `&start=1`. Sin esa guarda, un cron cada cinco minutos empezaría un análisis nuevo en cada llamada y no terminaría ninguno.

- **Salida:** texto plano con los hitos, apta para `wget -O /dev/null` y también para abrirla en el navegador. Las líneas de fichero individual se omiten para no generar miles de líneas.
- **Publicación:** el paso `publish` solo actúa si `PUBLISH_ENABLED` y `CRON_PUBLISH` están activados y el análisis lo lanzó el cron. Desde el panel la publicación sigue siendo un botón manual con confirmación.

11 ter. Generación de parches

El paso `patches` recorre los módulos con su propio cursor y presupuesto de tiempo, y escribe un fichero por módulo en `generated/informes/`:

```
<módulo>--v<version>--ps<destino>.diff
```

El nombre lleva la versión porque el mismo módulo puede existir con versiones distintas en tiendas distintas. Dentro, las rutas son las originales de la tienda (`a/modules/<módulo>/<fichero>`), de modo que el parche se aplica con `git apply` desde la raíz de PrestaShop.

Al publicar, `EcomCompatExporter::attachGeneratedPatches()` los coloca en `modulos/<módulo>/<version>/cambios-ps<destino>.diff`, conservando la separación por versión.

11 quater. Carpeta protegida y descargas

`EcomCompatFiles` gestiona `modules/ecom_compatible/generated/`, con tres subcarpetas: `informes/` (los `.diff`), `overrides/` (las propuestas) y `exportado/` (los JSON/MD y los ZIP).

Protección. Al instalar, y de nuevo cada vez que algo escribe ahí, se crea un `.htaccess` que cierra el acceso directo. Cubre Apache 2.2 y 2.4 a la vez, cada uno dentro de su `<IfModule>` para que la directiva que no corresponda no provoque un error 500:

```
<IfModule mod_authz_core.c>
    Require all denied
</IfModule>
<IfModule !mod_authz_core.c>
    Order deny,allow
    Deny from all
</IfModule>
```

Además hay `index.php` en cada subcarpeta. La única vía de descarga es `processDownload()` del controlador de administración, que válida permisos y token y sirve el fichero con `readfile()`.

Validación de la ruta. Es la superficie de ataque real, así que `EcomCompatFiles::resolve()` es deliberadamente restrictivo: exige exactamente dos segmentos (`carpeta/fichero`), la carpeta tiene que estar en una lista blanca, el nombre tiene que casar con `^[A-Za-z0-9._\-]{1,190}$`, se rechazan `..`, bytes nulos, dos puntos, rutas absolutas, `index.php` y `.htaccess`, y al final se compara el `realpath()` contra la base para que un enlace simbólico no pueda apuntar fuera. Hay una batería de once casos de ataque en las pruebas

— `../../../../../config/settings.inc.php`, `/etc/passwd`, `C:/Windows/win.ini`, byte nulo, subdirectorios— y los once quedan bloqueados.

Por qué la carpeta es plana. Guardar el árbol real en disco obligaría a que `resolve()` aceptara rutas de profundidad arbitraria, que es mucho más difícil de blindar. En disco los nombres se aplanan (`modulos__legacydemo__1.0.0__hallazgos.json`) conservando módulo y versión de forma legible, y al construir el ZIP se restaura el árbol (`modulos/legacydemo/1.0.0/hallazgos.json`), que es lo que el usuario abre en su ordenador.

11 quinquies. Regenerar y reiniciar

`ajaxProcessRegenerate()` rehace informes, parches y propuestas de override **a partir del histórico**, sin repetir el análisis. Tiene dos fases: `patches` (cursor sobre módulos, con presupuesto de tiempo, igual que el análisis) y `export` (vuelca el árbol del exportador a `exportado/` y regenera los overrides). El cursor lo lleva el cliente, no la base de datos, porque no hay que reanudarlo entre sesiones.

`ajaxProcessReset()` acepta un `scope`: `scan`, `files`, `cache`, `history` o `all`. Nunca toca la configuración ni los tokens, y `history` va aparte porque es el único irreversible.

11 sexies. Clasificación por proveedor y estado

`EcomCompatVendor` decide en qué sección del informe cae cada módulo. El estado manda sobre el proveedor: un módulo desactivado o sin instalar no se migra igual que uno en producción.

Estado (`resolveStatus`): `no_instalado` si hay carpeta pero no fila en la tabla `module`; `inactivo` si está instalado con `active = 0`; `activo` en el resto.

Proveedor (`resolve`), en este orden:

1. **PrestaShop** — el nombre está en `data/native-modules.json`, o el autor normalizado es exactamente `prestashop` / `prestashoppsa`. La comparación es exacta a propósito: hay autores como «Prestashop Templates» que **no** son PrestaShop y que en una comparación por prefijo se clasificarían mal.
2. **Tema** — el módulo aparece en las dependencias declaradas en `themes/<tema>/config/theme.yml` (secciones `modules`, `to_enable`, `to_disable`), tiene carpeta dentro de `themes/<tema>/modules/`, **o lo firma el mismo autor que el tema**. Esta última condición no es opcional: las plantillas comerciales traen familias enteras de módulos (`iqit*`, `st*`, `cz_*`) con el autor del tema pero sin declararlos como dependencia, y sin ella acaban en «terceros» cuando en realidad se actualizan con la plantilla. El YAML se lee con un analizador propio por líneas, sin dependencias externas.
3. **Propio** — el autor casa con la lista configurable `OWN_AUTHORS`.
4. **Sin autor** — `author` vacío en `config.xml` y en la clase principal.
5. **Tercero** — cualquier otro, agrupado por el `author` declarado.

`data/native-modules.json` contiene 69 nombres: la unión de los módulos empaquetados en las distribuciones reales 1.7.8.11, 8.1.7 y 8.2.7 más los que declara la Distribution API para 8.0 a 9.1. No cubre los módulos de PrestaShop que se instalan aparte (`paypal`, `ps_accounts`, `ps_mbo`, `gamification` ...), y por eso el respaldo por autor no es opcional: sin él esos cuatro caerían en «terceros».

Los valores se guardan en `ecomcompat_module` (`vendor` , `vendor_group` , `status_group`) para que el informe no tenga que recalcularlos, y el controlador agrupa en `getSections()` , que además tolera filas de análisis anteriores a esta versión recalculando la sección al vuelo.

Subdivisión por desarrollador. `splitByVendor()` parte cada sección en bloques por proveedor, ordenados por número de bloqueantes descendente. Con 47 módulos de terceros o 83 sin instalar, una tabla única no dice nada útil: lo accionable es ver cuántos vienen de cada proveedor y cuántos rompen, porque la gestión se hace proveedor a proveedor. La subdivisión solo se dibuja cuando hay más de un proveedor en la sección.

La tabla de módulos vive en `views/templates/admin/_modules_table.tpl` y se incluye tanto para una sección completa como para cada bloque de desarrollador, con la variable `$rows` .

Acordeón. Cada cabecera es un `<button>` real con `aria-expanded` y `aria-controls` , no un `div` con un manejador de clic: así funciona con teclado y con lector de pantalla sin código adicional. El cuerpo se oculta con la clase `is-collapsed` en lugar del atributo `hidden` , porque el CSS del back office aplica `display` a varios contenedores y ganaría al atributo.

Las secciones nacen cerradas y los bloques de desarrollador abiertos; ese estado inicial se declara en el HTML (`aria-expanded`) y el JavaScript lo respeta. El estado que el usuario elige se guarda en `localStorage` bajo `ecomcompat.abiertos` , dentro de `try/catch` para que en modo privado o con el almacenamiento lleno el acordeón siga funcionando sin persistencia. Los enlaces del índice llevan `data-open-section` y abren la sección antes de saltar: si no, el navegador salta a un panel cerrado y parece que no ha pasado nada.

El triángulo se dibuja con bordes CSS y no con una fuente de iconos, para que no dependa de que el tema del back office cargue FontAwesome.

11 septies. Exclusión mutua

`EcomCompatLock` impide que se ejecuten dos análisis a la vez. Sin él, dos pestañas del panel —o el cron corriendo mientras alguien pulsa «Analizar»— avanzan el mismo cursor simultáneamente: se duplican filas de módulo, se pierden ficheros por analizar y el progreso da saltos hacia atrás. Es un fallo silencioso y muy difícil de reproducir a mano.

La exclusión se apoya en un **UPDATE condicional**, que en MySQL es atómico a nivel de fila: la fila queda bloqueada mientras se evalúa el `WHERE` , así que solo una petición puede pasar de «libre» a «mía».

```
UPDATE ..._lock
SET owner = '<propietario>', expires = NOW() + INTERVAL 180 SECOND, date_upd = NOW()
WHERE lock_name = 'scan'
AND (owner = '' OR owner = '<propietario>' OR expires IS NULL OR expires <= NOW())
```

- El propietario es `scan:<id_scan>` , de modo que cualquier petición que continúe ese análisis lo renueva, pero un análisis distinto no puede empezar.
- Caduca a los 180 s: un proceso que muera no bloquea la herramienta para siempre.
- Se libera al terminar el análisis, al fallar, y con el botón «Reiniciar análisis», que hace una liberación forzada.

Tres reglas que vienen de un fallo real

En una tienda con 200 módulos el análisis moría en el primer lote con «Otro proceso está ejecutando un análisis ()» —y el paréntesis vacío delataba que **no había ningún otro proceso**. Eran tres defectos encadenados:

1. Ninguna fecha se compara en PHP. `expires` es una `DATETIME` escrita por MySQL con `NOW()`, es decir en la zona horaria del **servidor de base de datos**. Comparándola con `strtotime()` contra `time()` de PHP, cualquier alojamiento con MySQL en UTC y PHP en `Europe/Madrid` veía un cerrojo recién tomado como caducado dos horas antes. Ahora la caducidad la calcula MySQL y llega ya resuelta como una columna:

```
SELECT owner, (expires IS NOT NULL AND expires > NOW()) AS vivo FROM ..._lock WHERE lock_name = 'scan'
```

2. No se usa `Affected_Rows()`. En un `UPDATE`, MySQL cuenta filas **cambiadas**, no coincidentes. `start()` toma el cerrojo y el primer lote lo renueva dentro del mismo segundo: se reescriben exactamente los mismos valores, MySQL informa de 0 filas y el código deducía que había perdido la carrera. Quién tiene el cerrojo se comprueba releendo la fila; la atomicidad sigue viniendo del `UPDATE` condicional, la relectura solo dice quién ganó. Por lo mismo se descarta `INSERT IGNORE` + `Affected_Rows()`.

3. Un cerrojo averiado deja pasar. `Db::execute()` devuelve `false` sin lanzar excepción cuando la consulta falla, así que un `try/catch` no basta: hay que mirar el valor devuelto. Si la tabla no existe o la base de datos falla, `acquire()` devuelve `true` y el análisis continúa. `holder()` distingue internamente «libre» de «no se puede leer», pero hacia fuera devuelve cadena vacía: para quien llama, ambos significan «no hay nadie a quien respetar».

Sin caché SQL

Ninguna lectura de las tablas del módulo pasa por la caché de consultas de PrestaShop: `getRow()` y `executeS()` se llaman con `$use_cache = false`. El módulo escribe y relee sus propias tablas varias veces dentro de la misma petición —el cerrojo, el cursor del análisis, el anti-duplicado del histórico—, y una fila servida desde caché haría retroceder el cursor o duplicar hallazgos. La caché no aporta nada aquí: son consultas puntuales, no un camino caliente.

El lote nunca dura más que el cerrojo

`EcomCompatSettings::timeBudget()` topa el presupuesto de cada lote en la mitad del TTL (90 s). Sin ese tope, un `max_execution_time = 0` —lo normal por cron— con un presupuesto alto puesto a mano dejaría que el cerrojo caducara **a mitad del lote**, y otra pestaña podría arrancar encima.

Verificado con 22 asertos: exclusión mutua, renovación dentro del mismo segundo, desfase horario en ambos sentidos, caducidad real, liberación propia y ajena, liberación forzada, cerrojo averiado y saneado del identificador. Y de extremo a extremo con el reloj de MySQL adelantado dos horas a propósito: el análisis completo termina y suelta el cerrojo.

11 octies. Coste del análisis

Medido sobre un módulo real de 48 ficheros:

Operación	Coste
Carga de la base de conocimiento	18 ms (una vez por petición)
<code>listFiles()</code> de un módulo	64 ms la primera vez, más en repeticiones
<code>newAccumulator()</code> (clases propias por regex)	73 ms
<code>analyzeOneFile()</code>	≈ 30 ms por fichero

El recorrido del árbol de ficheros resultó ser un coste no despreciable que se repetía en cada lote. La lista se guarda ahora en el estado cuando el módulo tiene **2000 ficheros o menos**; por encima se vuelve a listar, porque arrastrar un blob de varios MB en cada lote saldría más caro. El efecto medido sobre el banco de pruebas de 659 ficheros: de 43 lotes y 52 s a **16 lotes y 12 s**, con resultados idénticos.

El presupuesto se comprueba **antes** de cada fichero, así que un fichero muy grande puede desbordarlo. Con el presupuesto real (8 s, y nunca más del 60 % del `max_execution_time`) el margen absorbe de sobra ese desbordamiento, y `MAX_FILE_SIZE` acota el tamaño de fichero analizable.

12. Seguridad

Resultado de la última auditoría, con el método usado en cada punto:

Comprobación	Método	Resultado
Inyección SQL	Búsqueda de valores interpolados sin <code>pSQL</code> / <code>bqSQL</code> /casteo	0 casos
XSS en plantillas	Búsqueda de <code>{ \$var }</code> sin <code>escape</code> y de <code>nofilter</code>	0 casos
XSS en JavaScript	Revisión de todos los <code>innerHTML</code>	Todos con <code>escapeHtml()</code> o literales
Path traversal	11 vectores de ataque probados	11 bloqueados
Funciones peligrosas	<code>eval</code> , <code>system</code> , <code>exec</code> , <code>unserialize</code> , <code>extract</code> ...	Ninguna
Inclusiones dinámicas	Búsqueda de <code>include</code> / <code>require</code> con variables	Ninguna: todas con ruta fija
Guarda <code>_PS_VERSION_</code>	Todos los <code>.php</code> salvo los <code>index.php</code>	0 sin ella
Filtración de secretos	Búsqueda de tokens en llamadas al registro	Ninguna: el token viaja en cabecera y nunca se registra
Permisos de los endpoints	Revisión uno a uno	Todo lo que muta o sale a internet exige permiso de edición

- Todos los `.php` empiezan con la cabecera de licencia y `if (!defined('_PS_VERSION_')) exit;` , sin líneas en blanco antes de `<?php` .
- `index.php` en todos los directorios de forma recursiva.
- El endpoint del cron es la única superficie pública: exige interruptor propio, token de 32 caracteres generado al azar y comparación con `hash_equals()` . Registra los intentos fallidos con su IP.
- Los ficheros de registro rotan a los 5 MB y se conservan como máximo 10, para que unos logs olvidados no llenen el disco del alojamiento.
- La descarga vacía **todos** los niveles de búfer de salida antes de servir el fichero: un solo byte previo corrompería la descarga.
- Entradas por `Tools::getValue()` y validación explícita; el token y el repositorio pasan por expresión regular antes de guardarse.
- Salida escapada en Smarty con `|escape:'html':'UTF-8'` ; sin `nofilter` .
- El JavaScript recibe la configuración por atributos `data-*` , no por bloques inline.
- Los endpoints AJAX comprueban permiso de edición del empleado.
- SQL: `pSQL()` para cadenas, `bqSQL()` para identificadores, enteros casteados.

13. Limitaciones conocidas

- Los mapas del núcleo proceden de 1.7.8.11, 8.1.7, 8.2.7 y 9.1.1. Un destino 8.0 se evalúa con el mapa de 8.1 y un destino 9.0 con el de 9.1: el resultado es ligeramente conservador, nunca permisivo.
- El análisis es estático. No detecta llamadas construidas en tiempo de ejecución (`call_user_func('Tools::' . $x)`), ni incompatibilidades que solo se manifiestan con datos concretos.
- Las propiedades dinámicas de PHP 8.2 solo se avisan en clases sin herencia y sin `__set()`, para no inundar el informe.
- El análisis del tema compara con las plantillas del módulo instalado, no con las del tema de destino.
- Los módulos cifrados o con `ionCube` no se pueden analizar.

Ecom Compatible — Guía de desarrollo

Cómo extender, mantener y regenerar el módulo.

1. Añadir una regla nueva

Todas las reglas viven en `data/*.json`. **No hay que tocar PHP** para añadir la mayoría.

Función de PHP eliminada u obsoleta

`data/rules-php.json` → `functions_removed` O `functions_deprecated`:

```
"nombre_funcion": {
  "php": "8.3",
  "severity": "blocker",
  "replacement": "funcion_nueva()",
  "note": "Por qué deja de funcionar y qué pasa si no se cambia."
}
```

`php` es la versión a partir de la cual aplica. La regla solo se evalúa si el destino de PHP configurado es igual o superior.

Regla por expresión regular

`data/rules-php.json` → `regex`, O `data/rules-prestashop.json` → `code_rules`:

```
{
  "id": "identificador_unico",
  "php": "8.2",
  "severity": "warning",
  "pattern": "expresion\\s+regular",
  "title": "Título corto que se ve en la tabla",
  "note": "Explicación de la consecuencia real.",
  "fix": "Instrucción concreta de qué escribir en su lugar."
}
```

El patrón se aplica **línea a línea**, con las líneas de comentario simple ya descartadas. Se delimita con `#`, así que hay que escapar las almohadillas literales. `scope` admite `php` (por defecto) y `php_namespaced` (solo ficheros con `namespace`).

En `rules-prestashop.json` se usa `since` en lugar de `php`: la versión de PrestaShop a partir de la cual aplica.

Regla de plantilla

`data/rules-smarty.json` → `rules`. Mismos campos, con `since` de PrestaShop.

Sustitución recomendada

`data/rules-replacements.json`. Aparece en la columna «Que cambiar» cuando el motor detecta un símbolo eliminado:

```
"Clase::método": {
  "since": "9.0",
  "replacement": "Clase::metodoNuevo()",
  "note": "Por qué cambió.",
  "example_before": "$x = Clase::método($a);",
  "example_after": "$x = Clase::metodoNuevo($a);"
}
```

Reescritura mecánica (entra en los `.diff`)

`data/rules-rewrites.json`. **Solo sustituciones 1 a 1 que no cambian la semántica.** Si hace falta criterio humano, va en `rules-replacements.json`, no aquí.

```
{ "id": "identificador", "since": "9.0", "search": "Texto::viejo(", "replace": "Texto::nuevo(",
  "note": "..."}
}
```

Si `search` empieza por letra o `_`, se antepone automáticamente la frontera `(?![A-Za-z0-9_\\$>])`. Para patrones que empiezan por `-`, `$` o `>` no se aplica frontera: tenlo en cuenta al elegir el texto.

2. Regenerar los mapas del núcleo

Los ficheros `core-diff-*.json`, `core-classes-*.json` y `hooks-*.json` se generan comparando código fuente real. Para añadir un destino nuevo (por ejemplo 9.2):

1. Descarga la versión y descomprime al menos `classes/`, `controllers/`, `src/Core/`, `src/Adapter/` e `install/data/xml/hook.xml`.
2. Ejecuta el extractor de símbolos sobre cada versión:

```
bash php extract_symbols.php <raiz_prestashop> sym-9.2.json classes controllers src/Core src/Adapter
```

3. Ejecuta el constructor de datos, que calcula el diff contra la unión de las versiones anteriores y escribe en `data/`.
4. Añade la versión a `EcomCompatSettings::targetVersions()` y el mapeo en `EcomCompatSettings::resolveDataset()`.

Aviso al tocar el extractor: el seguimiento de la profundidad de llaves tiene que contar `T_CURLY_OPEN` y `T_DOLLAR_OPEN_CURLY_BRACES`. La interpolación `"{$var}"` abre con un token array y cierra con un `}` literal; si no se cuenta la apertura, el contador se descuadra y se pierden métodos sin ningún error visible. Comprueba siempre después de regenerar que `HookCore::exec` está presente en el mapa:

```
$m = json_decode(file_get_contents('sym-9.1.json'), true);  
var_dump(isset($m['classes']['HookCore']['methods']['exec'])); // debe ser true
```

3. Probar sin una tienda

El motor no depende de PrestaShop en ejecución: basta con definir las constantes y cuatro clases mínimas (`Tools` , `Configuration` , `Db` , `Context`) y las funciones `pSQL()` y `bqSQL()` . Con eso se puede ejecutar `EcomCompatCoreAnalyzer` , `EcomCompatPatchBuilder` , `EcomCompatOverrideBuilder` y `EcomCompatExporter` contra una carpeta de módulos cualquiera.

Para que el banco de pruebas encuentre los datos hay que extender `EcomCompatKnowledge` y sobrescribir `getDataDir()` . Por eso `load()` usa `static::getDataDir()` y no `self::` .

Ciclo de validación recomendado tras cualquier cambio:

1. Lint con PHP 7.4 y 8.3 de todos los `.php` .
2. Analizar un conjunto de módulos oficiales de PrestaShop 1.7.8 → deben salir **0 bloqueantes**. Si aparece alguno, es un falso positivo.
3. Analizar un módulo con incompatibilidades inyectadas → deben detectarse todas.
4. Generar un `.diff` y validarlo con `git apply --check` .
5. Lintar el override generado.

4. Reglas de estilo del módulo

El módulo tiene que ejecutarse en la tienda **antigua**, que es la que se va a migrar. Eso condiciona el código:

- Sin propiedades tipadas, sin tipos de retorno, sin `match` , sin funciones flecha, sin `??=` , sin promoción de propiedades en el constructor.
- Sin `namespace` en la clase principal ni en los overrides.
- Sin `declare(strict_types=1)` : el núcleo legacy pasa argumentos sin castear.
- Sin dependencias de Composer. Carga con `require_once` .
- Nunca `$variable::class` ; usar `get_class()` .
- `\Context::getContext()` con barra invertida inicial en ficheros con namespace.
- Cabecera de licencia pegada a `<?php` , sin líneas en blanco, y `if (!defined('_PS_VERSION_')) exit;` justo después.
- `index.php` en cada directorio nuevo.

El propio módulo se audita a sí mismo: si aparece en su informe con bloqueantes, hay algo que corregir.

5. Añadir un paso al análisis

`EcomCompatRunner` encadena pasos guardando el estado en `ecomcompat_scan`:

```
prepare → modules → overrides → theme → patches → finish → publish → done
```

Para añadir uno:

1. Declara la constante `STEP_LOQUESEA`.
2. Escribe `stepLoQueSea(array $scan)` devolviendo `['ok', 'done', 'progress', 'step', 'console', 'message']`.
3. Enlázalo en el `switch` de `step()` y haz que el paso anterior actualice `step` al nuevo valor.
4. Llama a `EcomCompatFinding::flush()` antes de devolver.

El porcentaje es orientativo: módulos 0–70, overrides 80, tema 84, parches 90, cierre 97, publicación 100.

Todo paso que pueda recorrer muchos elementos debe llevar presupuesto de tiempo, con este patrón:

```
$budget = EcomCompatSettings::timeBudget();
$started = microtime(true);
while ($cursor < $total) {
    if (microtime(true) - $started >= $budget) { break; }
    // ... procesar UN elemento ...
    ++$cursor;
}
$this->updateScan($idScan, ['cursor_index' => $cursor, 'step' => $cursor >= $total ? SIGUIENTE : ACTUAL]);
```

Nunca proceses un módulo entero de golpe: hay módulos de terceros con miles de ficheros.

No guardes listas grandes en la columna `state`. Se escribe y se relee en cada lote. `listFiles()` devuelve orden alfabético estable, así que basta con el cursor numérico y recalculas la lista. En `state` va solo lo que no se puede deducir de un fichero suelto: clases propias del módulo, hooks registrados y declarados, banderas de estructura.

console

Cada paso devuelve un array `console` con líneas `['t' => tipo, 'm' => sujeto, 'x' => texto, 'sev' => severidad]` que el navegador vuelca en el panel y el cron escribe en texto plano. Los tipos `fichero` se omiten en el cron para no generar miles de líneas.

6. Añadir una fuente remota

`EcomCompatRemoteAnalyzer::analyzeModule()` prueba las fuentes en orden y se queda con la primera concluyente. Para añadir otra:

1. Escribe `queryLoQueSea(array $module)` devolviendo `['status', 'latest', 'compatible', 'source', 'url', 'message']`.
2. Encájala en la cadena de `analyzeModule()`.
3. Usa siempre `EcomCompatHttp::getJson()` : respeta el interruptor general, la caché y el límite de peticiones por lote.
4. Añade su interruptor propio en `EcomCompatSettings`.

Ninguna fuente nueva debe hacer peticiones fuera del análisis manual.

6 bis. Trampas de PrestaShop ya pisadas

Están documentadas porque cuestan horas de encontrar y no dan ningún error:

Síntoma	Causa	Solución aplicada
La página se pinta pero los botones no hacen nada	<code>addCSS()</code> / <code>addJS()</code> traen <code>\$check_path = true</code> : si la URI no resuelve contra el sistema de ficheros, PrestaShop descarta el asset en silencio	Llamarlos con <code>\$check_path = false</code> , y pintar un aviso en la plantilla que el propio JS elimina al cargarse
El JS se carga pero no engancha nada, y el aviso de «JS no cargado» se queda puesto	El back office imprime los <code>js_files</code> en <code>admin/themes/*/template/header.tpl</code> , es decir dentro de <head> : el script se ejecuta antes de que exista el contenido, <code>getElementById()</code> devuelve <code>null</code> y un <code>return</code> temprano lo aborta todo en silencio	Envolver la inicialización en <code>boot()</code> y llamarla con <code>DOMContentLoaded</code> cuando <code>document.readyState === 'loading'</code>
Se pierden métodos al tokenizar	<code>"{\$var}"</code> abre con el token <code>T_CURLY_OPEN</code> (array) y cierra con un <code>}</code> literal (string): el contador de llaves se descuadra	Contar <code>T_CURLY_OPEN</code> y <code>T_DOLLAR_OPEN_CURLY_BRACES</code> como apertura
Las filas van a una tabla sin prefijo	El sexto argumento de <code>Db::insert()</code> es <code>\$add_prefix</code> , no una bandera de caché	Dejarlo en <code>true</code>
<code>git apply</code> rechaza el parche	<code>preg_split</code> deja un elemento vacío por el salto de línea final y la cabecera <code>@@</code> cuenta una línea de más	<code>EcomCompatPatchBuilder::splitLines()</code> lo descarta
Todo HTTPS falla con <code>unable to get local issuer certificate</code>	<code>curl.cainfo</code> vacío en el <code>php.ini</code> del alojamiento	Usar el <code>cacert.pem</code> que distribuye PrestaShop, sin desactivar la verificación
La Git Data API devuelve 409 en un repositorio nuevo	Un repositorio sin ningún commit rechaza blobs, árboles y commits	Sembrar el primer commit con la Contents API y seguir
El controlador de front del módulo da 404	El despachador compone <code><nombre_modulo> . <controlador> . 'ModuleFrontController'</code> sin CamelCase	Los nombres de clase de PHP no distinguen mayúsculas, así que <code>Ecom_CompatibleCronModuleFrontController</code> resuelve
«Otro proceso está ejecutando un análisis ()»,	Tres cosas a la vez: la caducidad del cerrojo se comparaba con el reloj de PHP contra una <code>DATETIME</code> escrita por MySQL; <code>Affected_Rows()</code> daba 0 al renovar dentro	Comparar fechas dentro de SQL, comprobar la propiedad relejando la fila, y devolver <code>true</code> si el cerrojo no se puede usar (ver DOC-TECNICO § 11 septies)

Síntoma	Causa	Solución aplicada
con el paréntesis vacío	del mismo segundo porque MySQL cuenta filas <i>cambiadas</i> ; y un fallo de la propia tabla hacía fallar cerrado en vez de abierto	
El cursor del análisis retrocede o se duplican hallazgos	<code>Db::getRow()</code> y <code>Db::executeS()</code> sirven desde la caché de consultas de PrestaShop cuando está activada, y el módulo relee sus propias tablas dentro de la misma petición	Llamarlas siempre con <code>\$use_cache = false</code> sobre las tablas del módulo
Una consulta de escritura «no hace nada» pero tampoco lanza excepción	<code>Db::execute()</code> devuelve <code>false</code> en vez de lanzar cuando <code>_PS_MODE_DEV_</code> está apagado	Mirar el valor devuelto; un <code>try/catch</code> no basta

7. Qué no hacer

- **El análisis nunca debe dispararse desde un hook ni desde la carga de una página.** Solo desde el botón del panel o desde el endpoint del cron, que exige token y su propio interruptor.
- **No copiar overrides generados a `/override` automáticamente.** Se escriben en `generated/overrides/` y los instala el comerciante.
- **No publicar en GitHub sin confirmación explícita**, ni activar por defecto la inclusión de `.diff` u overrides: contienen código real de la tienda.
- **No usar `installOverrides()` en este módulo.** Es una convención del proyecto, no una limitación de PrestaShop: `Module::addOverride()` fusiona miembro a miembro y convive con otros módulos sin destruir nada (ver DOC-TECNICO § 5). Aquí se evita simplemente porque el módulo no debe modificar el núcleo de la tienda que audita. Si alguna vez lo necesitara, copia manual desde `override_v8/` comprobando que el destino no existe, y borrado de `var/cache/*/class_index.php`.
- **No subir una regla `blocker` sin verificarla** contra módulos oficiales de PrestaShop: un falso bloqueante hace que el comerciante deje de confiar en todo el informe.
- **No comparar nunca una fecha de MySQL con el reloj de PHP.** `strtotime($fila['expires']) < time()` da por caducado lo que no lo está en cuanto MySQL y PHP tienen zonas horarias distintas, que es lo normal en alojamiento compartido. La comparación se hace en SQL.
- **No usar `Affected_Rows()` para decidir quién ganó una carrera.** MySQL cuenta filas *cambiadas*: reescribir los mismos valores devuelve 0 aunque la fila coincidiera. Releer.
- **Ningún lote puede durar más que el TTL del cerrojo.** `timeBudget()` lo topa en la mitad; si se sube el TTL, ese tope sube solo.

8. Depuración

Activa «Guardar logs» en la configuración. Se escribe en `modules/ecom_compatible/logs/ecom_compatible-AAAA-MM-DD.log` con nivel, identificador de petición y canal (`runner` , `knowledge` , `http` , `github` , `db` , `inventory` , `template` , `override_builder`).

Los errores se registran siempre que la carpeta exista, aunque los logs estén desactivados. Rotación automática a los 5 MB.